LAB 4

Sama nabil

SE2

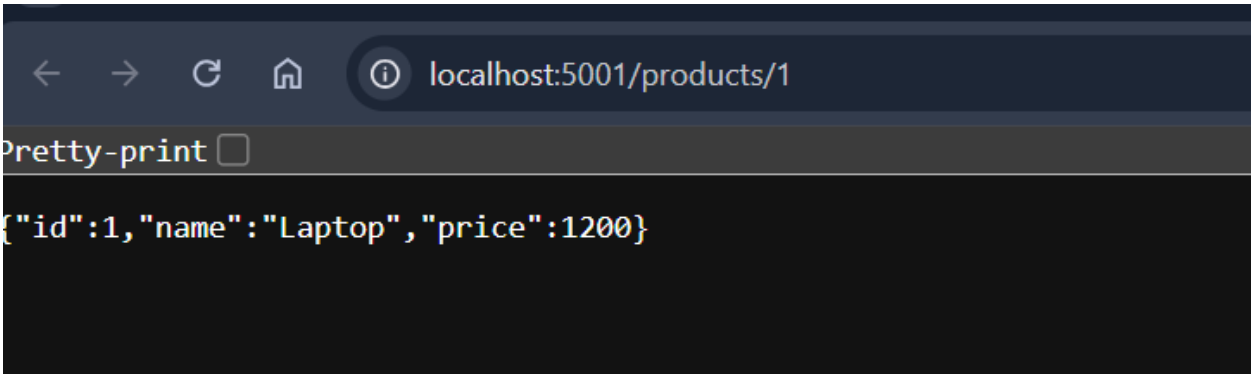
docker compose up

```
 ✔Image lab4-order-service   Built                                                                    2.1
 ✔Image lab4-product-service Built                                                                    2.1
 ✔Network lab4_default       Created                                                                  0.0
 ✔Container product-service  Created                                                                  0.1
 ✔Container order-service    Created                                                                  0.1
Attaching to order-service, product-service
product-service  |  * Serving Flask app 'app'
product-service  |  * Debug mode: off
product-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
I server instead.
product-service  |  * Running on all addresses (0.0.0.0)
product-service  |  * Running on http://127.0.0.1:5001
product-service  |  * Running on http://172.20.0.2:5001
product-service  | Press CTRL+C to quit
order-service    |  * Serving Flask app 'app'
order-service    |  * Debug mode: off
order-service    | WARNING: This is a development server. Do not use it in a production deployment. Use a production WS
I server instead.
order-service    |  * Running on all addresses (0.0.0.0)
order-service    |  * Running on http://127.0.0.1:5002
order-service    |  * Running on http://172.20.0.3:5002
order-service    | Press CTRL+C to quit
product-service  | 172.20.0.1 - - [23/Apr/2026 03:23:34] "GET /health HTTP/1.1" 200 -
product-service  | 172.20.0.1 - - [23/Apr/2026 03:23:42] "GET /products/1 HTTP/1.1" 200 -
order-service    | 172.20.0.1 - - [23/Apr/2026 03:23:50] "POST /orders HTTP/1.1" 400 -
product-service  | 172.20.0.3 - - [23/Apr/2026 03:24:17] "GET /products/1 HTTP/1.1" 200 -
order-service    | 172.20.0.1 - - [23/Apr/2026 03:24:17] "POST /orders HTTP/1.1" 201 -
v View in Docker Desktop   o View Config   w Enable Watch   d Detach
```

SUCCESS request

```
PS C:\Users\sama> curl.exe http://localhost:5001/health
{"service":"product-service","status":"up"}
```

```
PS C:\Users\sama> curl.exe http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}
```



localhost:5001/health

Pretty-print


```
{
  "service": "product-service",
  "status": "up"
}
```


FAILURE test

```
PS C:\Users\sama> Invoke-RestMethod -Method POST -Uri "http://localhost:5002/orders" -ContentType "application/json" -Bo
dy '{"product_id":1,"quantity":2}'

message       product quantity total_price
-------       ------- -------- -----------
Order created Laptop         2        2400
```

```
C:\Users\sama>docker stop product-service
product-service

C:\Users\sama>curl.exe -X POST "http://localhost:5002/orders" -H "Content-Type: application/json" -d "{\"product_id\":1,
\"quantity\":2}"
{"error":"product-service unavailable"}

C:\Users\sama>
```

Optional extension tasks

payment-service

```
PS C:\Users\sama> curl.exe -X POST http://localhost:5002/orders -H "Content-Type: application/jso
1,""quantity"":2}"
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>The browser (or proxy) sent a request that this server could not understand.</p>
PS C:\Users\sama> Invoke-RestMethod -Uri "http://localhost:5002/orders" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"product_id":1,"quantity":2}'


message        : Order created + paid
payment_status : @{message=Payment successful; paid_amount=2400}
product        : Laptop
quantity       : 2
total_price    : 2400
```

service failure

```
PS C:\Users\sama> docker stop product-service
product-service
PS C:\Users\sama>  curl.exe -X POST http://localhost:5002/orders -H "Content-Type: applicati
:1,""quantity"":2}"
<!doctype html>
<html lang=en>
<title>400 Bad Request</title>
<h1>Bad Request</h1>
<p>The browser (or proxy) sent a request that this server could not understand.</p>
PS C:\Users\sama> Invoke-RestMethod -Uri "http://localhost:5002/orders" `
>> -Method POST `
>> -ContentType "application/json" `
>> -Body '{"product_id":1,"quantity":2}'
Invoke-RestMethod : {"error":"product-service unavailable"}
At line:1 char:1
+ Invoke-RestMethod -Uri "http://localhost:5002/orders" `
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [
   eption
    + FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.In
```

QUESTIONS

1. Benefits of microservices over a monolith

This lab shows microservices benefits by splitting the system into two services: product-service and order-service. Each service has one responsibility, which makes the system easier to manage and update. They can also be developed and scaled independently without affecting each other.

---

2. New complexities introduced

Splitting the system adds complexity because services now communicate over the network instead of local calls. We also need to manage service URLs using environment variables and use Docker Compose to run everything. Debugging is harder since issues can come from either service or the connection between them.

---

3. What happens if latency increases or a service becomes slow

If product-service becomes slow, order-service will also slow down because it depends on it. Requests may time out or fail after retries, returning errors like 503. This shows that even in microservices, services are still strongly dependent on each other.

---

4. 12-factor app principles used

This lab shows several 12-factor principles: using environment variables for configuration, stateless services, port binding, and explicit dependencies in requirements.txt. Each service also runs in its own Docker container, keeping them independent.

Short report/reflection

In this lab, a simple microservices architecture was developed that uses Python and Flask. It comprises of three services: product-service, order-service, and the new payment-service. Each service was built as its own container and they communicate via HTTP protocol.

The primary benefit seen in this experiment is separation of concerns since each service deals with its unique responsibility like products, orders, and payment. Adding another service was relatively easy without the need for significant modifications to other services.

Nevertheless, there are limitations associated with microservices architecture due to the complex nature of its implementation. Dependencies are introduced since there is network communication involved which is not always reliable. This was seen in the experiment where by stopping the product-service led to order-service responding with 503 error. The use of synchronous communication implies that there are inter-dependent runtime services.

Finally, in terms of cloud native principles, the experiment utilized environment variables, statelessness in the services, and used containerization.